Matematički fakultet univerziteta u Beogradu

Planiranje putanje robota korišćenjem algoritma optimizacije kolonije mrava

Jovan Milanović

1. Uvodni deo

Planiranje putanje predstavlja fundamentalni problem mnogih oblasti. Cilj je pronaći optimalnu ili skoro optimalnu putanju od početne do krajnje tačke u datom okruženju, izbegavajući prepreke. Efikasno rešavanje ovog problema je ključno za autonomiju mobilnih robota u različitim primenama, od industrijske automatizacije do istraživanja nepoznatih terena.

Problem pronalaženja najkraće putanje u složenim okruženjima je NP-težak, što znači da klasični algoritmi često nisu praktični zbog eksponencijalnog porasta vremena izvršavanja sa veličinom problema. Zbog toga se u praksi često pribegava metaheuristikama, koje su u stanju da pronađu dovoljno dobra rešenja u prihvatljivom vremenskom roku.

Jedna od takvih metaheuristika je Optimizacija kolonijom mrava (eng. *Ant Colony Optimization, ACO*). Inspirisana ponašanjem mrava u potrazi za hranom, ACO je populacioni algoritam koji koristi indirektnu komunikaciju između agenata (mrava) putem hemijskih tragova (feromona). Mravi koji pronađu kraće putanje ostavljaju jači feromonski trag, što privlači druge mrave da slede istu putanju. Ovaj mehanizam pozitivne povratne sprege omogućava koloniji da konvergira ka optimalnom rešenju.

Ovaj projekat se bavi implementacijom i vizuelizacijom ACO algoritma za rešavanje problema planiranja putanje za robota u 2D statičkom okruženju sa preprekama.

2. Opis rešenja

Rešenje je realizovano u programskom jeziku C++ i sastoji se od nekoliko ključnih komponenata: modela okruženja, implementacije grafa, samog ACO algoritma i modula za vizuelizaciju.

2.1. Modelovanje okruženja i grafa

Okruženje je predstavljeno kao 2D mreža (grid), gde svaka ćelija može biti prohodna (.), prepreka (#), početna (S) ili krajnja (E) tačka. Ova reprezentacija je implementirana u klasi Map.

Da bi se algoritam mogao primeniti, mreža se transformiše u težinski neusmereni graf, što je zadatak klase Graph.

- **Čvorovi :** Svaka prohodna ćelija u mreži postaje jedan čvor u grafu.
- **Grane:** Grane povezuju susedne prohodne ćelije. Implementacija podržava 8-smerno kretanje, što znači da je svaki čvor povezan sa svojim susedima po horizontali, vertikali i dijagonali.

- **Težine grana:** Težine grana predstavljaju Euklidsku udaljenost između centara susednih ćelija. Za kretanje po horizontali i vertikali težina je 1.0, dok je za dijagonalno kretanje težina $\sqrt{2} \approx 1.414$.

2.2. Implementacija ACO algoritma

Srž rešenja je klasa ACO, koja implementira logiku optimizacije kolonijom mrava. Algoritam funkcioniše kroz seriju iteracija, a u svakoj iteraciji određeni broj veštačkih "mrava" konstruiše putanju.

Parametri algoritma (ACO::Params):

- numAnts: Broj mrava u jednoj iteraciji.
- iterations: Ukupan broj iteracija algoritma.
- alpha (α): Faktor koji kontroliše uticaj feromonskog traga.
- beta (β): Faktor koji kontroliše uticaj heurističke informacije (udaljenosti).
- evaporation: Koeficijent isparavanja feromona.
- depositBestOnly: Logička vrednost koja određuje da li feromon ostavlja samo globalno najbolji mrav (Elitistički pristup) ili svi mravi koji su pronašli put.

Konstrukcija putanje: Svaki mrav započinje putanju od početnog čvora S. U svakom koraku, mrav bira sledeći čvor na osnovu verovatnoće tranzicije, koja zavisi od količine feromona na grani i heurističke informacije. Verovatnoća da mrav pređe iz čvora i u čvor j data je formulom: $P_{ij} = \frac{[\tau_{ij}]^{\alpha} * [\eta_{ij}]^{\beta}}{\Sigma_{k \in N_i} [\tau_{ik}]^{\alpha} * [\eta_{ik}]^{\beta}}$ gde je:

- τ_{ij} količina feromona na grani (i, j).
- η_{ij} heuristička informacija, definisana kao $\frac{1}{d_{ij}}$, gde je d_{ij} težina grane (udaljenost). Ovo favorizuje kraće segmente puta.
- N_i skup neposećenih suseda čvora i.

Izbor sledećeg čvora se vrši metodom "ruletskog točka" (*roulette wheel selection*), gde svaka moguća grana ima šansu da bude izabrana srazmerno njenoj izračunatoj verovatnoći. Da bi se izbeglo formiranje ciklusa, svaki mrav pamti listu posećenih čvorova.

Ažuriranje feromona: Nakon što svi mravi završe svoje putanje u jednoj iteraciji, feromonski tragovi se ažuriraju u dva koraka:

1. **Isparavanje (Evaporation):** Količina feromona na svim granama se smanjuje za određeni procenat: τ_{ij} = (1 - evaporation) * τ_{ij}. Ovo omogućava algoritmu da "zaboravi" loše odluke i istražuje nove delove prostora rešenja. Postavljen je i minimalni nivo feromona kako bi se izbegla potpuna stagnacija.

2. **Polaganje (Deposition):** Mravi koji su uspešno stigli do cilja ostavljaju feromon na granama koje čine njihovu putanju. Količina položenog feromona je obrnuto srazmerna dužini (ceni) putanje: $\Delta\tau_{ij} = \frac{1}{L_k}$, gde je L_k ukupna cena putanje k-tog mrava. Na ovaj način, kraće putanje bivaju više "nagrađene" i postaju privlačnije za buduće mrave.

2.3. Vizuelizacija

Za lakšu analizu i razumevanje ponašanja algoritma, razvijen je modul za vizuelizaciju (Visualizer) korišćenjem SFML biblioteke. Vizuelizator iscrtava mapu sa preprekama, početnom i krajnjom tačkom. Glavna funkcionalnost je prikaz najbolje pronađene putanje za svaku pojedinačnu iteraciju. Korisnik može interaktivno da se kreće kroz iteracije pomoću dugmadi ili unosom broja iteracije, što omogućava praćenje procesa konvergencije algoritma od početnih, nasumičnih putanja do finalnog, optimizovanog rešenja.

2.4 Proširenje na problem trgovačkog putnika (TSP)

Pored pronalaženja najkraćeg puta između dve tačke, sistem je proširen na rešavanje NP-teškog problema: **problem trgovačkog putnika na grafu sa preprekama** (*Multi-Waypoint TSP*).

Definicija problema. Na mapi je dato N tačaka prolaska (čvorova *W*). Potrebno je pronaći redosled obilaska svih tačaka koji minimizuje ukupnu cenu puta. Broj mogućih redosleda iznosi N!, što problem čini eksponencijalnog rasta — na primer, za N = 8 postoji 8! = 40.320 mogućih rasporeda. Problem je NP-težak redukcijom na klasični TSP.

Implementirane su dve varijante:

- **Klasični TSP** — mreža ne sadrži krajnju tačku *E*; mrav se vraća u polaznu tačku.
- **Putni TSP** — mreža sadrži *E*; tura završava u fiksnoj krajnjoj tački.

Dvostepeni pristup. Direktna primena ACO-a na svim čvorovima mreže bila bi neefikasna zbog veličine prostora pretrage. Umesto toga, koristi se dvostepeni pristup:

1. **Predobrada (Dijkstrin algoritam).** Pre pokretanja ACO-a, Dijkstrin algoritam se pokreće iz svakog ključnog čvora (polazište, tačke prolaska, odredište). Na taj način se gradi matrica troškova K×K, gde K predstavlja broj ključnih čvorova. Složenost ove faze je O(K · (V + E) log V), gde V i E označavaju broj čvorova i grana mreže. Čuva se i matrica prethodnika, koja omogućava rekonstrukciju konkretnog puta na mreži između svaka dva ključna čvora.
2. **ACO na apstraktnom grafu.** ACO se zatim pokreće na potpunom grafu od K čvorova — ne na originalnoj mreži. Svaki mrav gradi turu obilaska svih tačaka prolaska. Selekcija sledećeg čvora vrši se probabilistički. Za trenutni čvor *i* i kandidata *j* iz skupa neposećenih čvorova, verovatnoća izbora iznosi: $p(i \rightarrow j) = (\tau_{ij}^{\alpha} \cdot \eta_{ij}^{\beta}) / \Sigma_k (\tau_{ik}^{\alpha} \cdot \eta_{ik}^{\beta})$, gde je τ_{ij} nivo feromona na grani (i, j), $\eta_{ij} = 1/d_{ij}$ heuristička informacija (recipročna vrednost udaljenosti između ključnih čvorova i i j), dok su α i β parametri koji kontrolišu relativni uticaj feromona i heuristike.

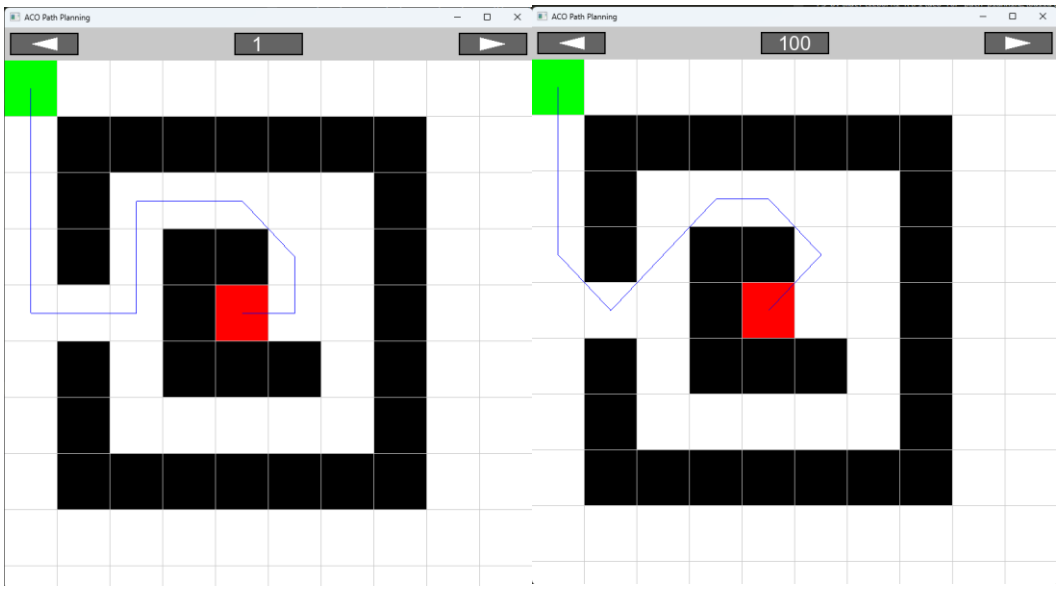
3. . Matrica feromona dimenzija K×K ažurira se isparavanem i depozicijom — pri čemu se koristi strategija *best-only* depozicije, gde feromont deponuje samo globalno najbolja tura pronađena do tekuće iteracije.

3. **Rekonstrukcija puta.** Nakon pronalaženja optimalnog redosleda obilaska, konkretni putevi između uzastopnih ključnih čvorova (preračunati Dijkstrom) spajaju se u jedinstven put na mreži koji se prikazuje u vizualizatoru.

3. Eksperimentalni rezultati

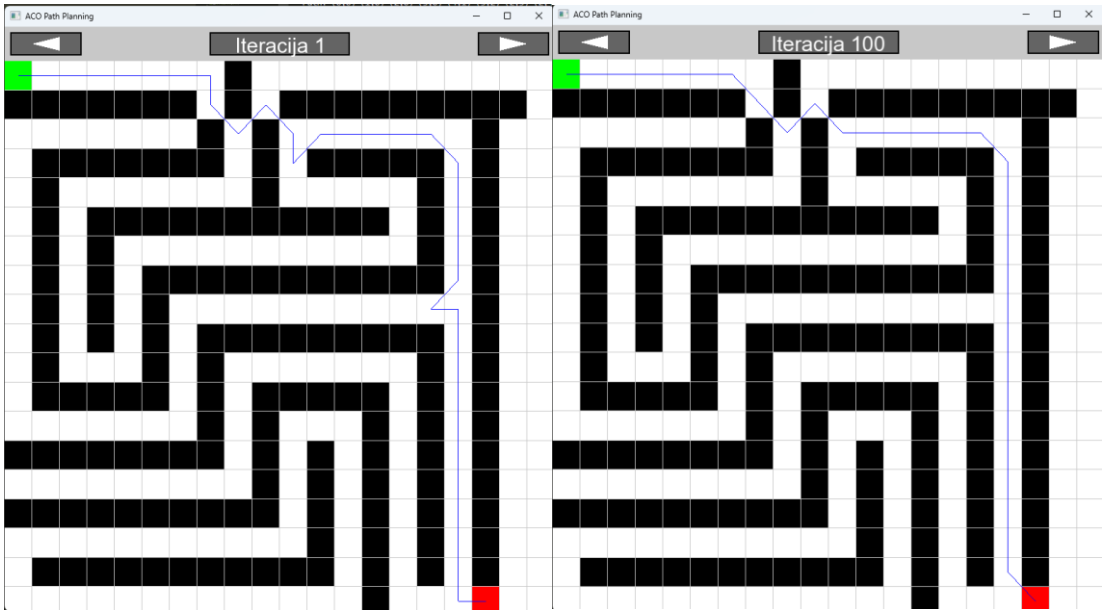
Algoritam je testiran na tri različite mape, rastuće složenosti, kako bi se procenile njegove performanse. Korišćeni su podrazumevani parametri (alpha=1.0, beta=2.0, evaporation=0.5), osim za najsloženiju mapu gde su parametri prilagođeni radi boljih rezultata.

3.1: Jednostavna mapa(10x10)



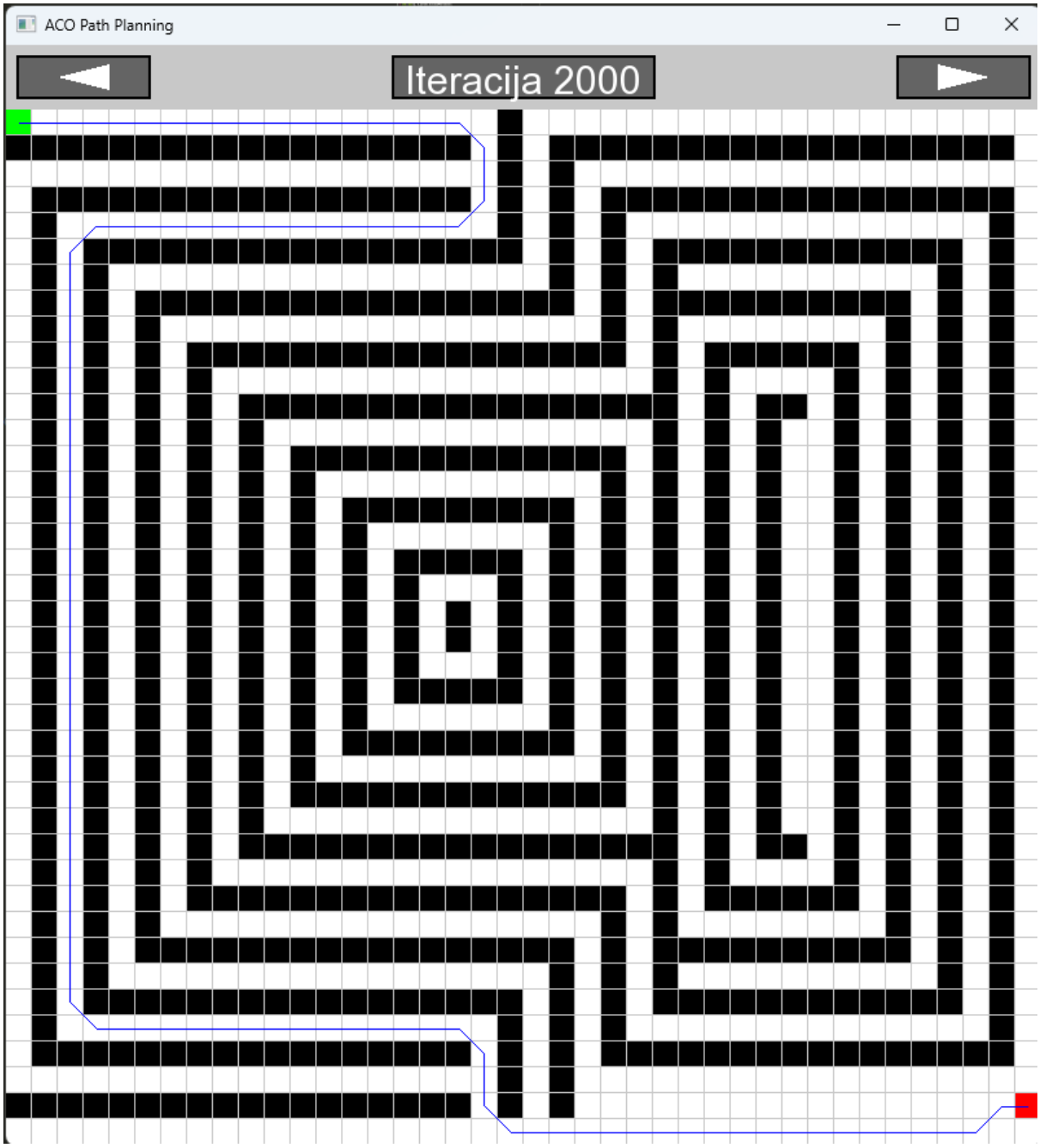
- **Opis:** Mala mapa sa nekoliko jednostavnih prepreka.
- **Parametri:** numAnts=64, iterations=100, depositBestOnly=true.
- **Rezultati:** Algoritam veoma brzo (oko 150ms) pronalazi optimalnu putanju. Konvergencija je brza, a vizuelizacija pokazuje da mravi već nakon malog broja iteracija dominantno prate najbolju rutu.

3.2 Srednja mapa (20x20)



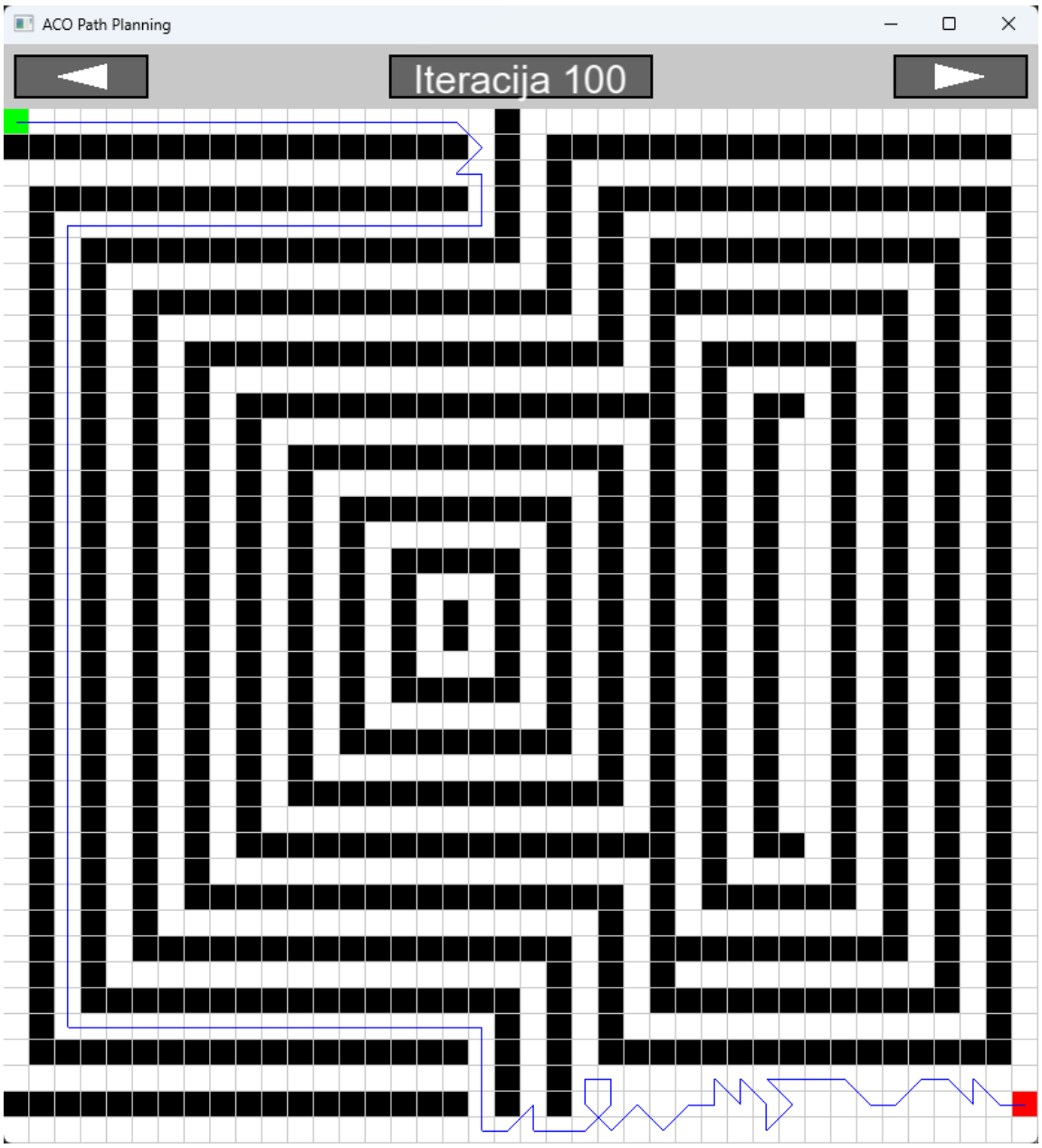
- **Opis:** Mapa srednje veličine sa strukturom koja podseća na lavirint.
- **Parametri:** numAnts=64, iterations=100, depositBestOnly=true.
- **Rezultati:** Vreme izvršavanja je nešto duže, oko 300ms. Pronađena putanja je visokog kvaliteta i vizuelno deluje optimalno.

3.3 Složena mapa(40x40)



- **Opis:** Velika i kompleksna mapa sa dugačkim, uskim hodnicima, dizajnirana da bude izazov za algoritme pretrage.
- **Parametri:** numAnts=64, iterations=2000, depositBestOnly=true, evaporation=0.1, minimumPheromone=0.0001.
- **Obrazloženje parametara:** Broj iteracija je značajno povećan da bi se algoritmu dalo dovoljno vremena da istraži veliki prostor. Evaporacija je smanjena, jer postoji velika mogućnost da će se mravi "zaglaviti" u nekim iteracijama (što je u redu, jer je broj iteracija povećan), pa ne želimo da se putanje brzo zaboravljaju. Minimalna feromonska vrednost je značajno smanjena, jer se očekuje da će putanja biti skupa,

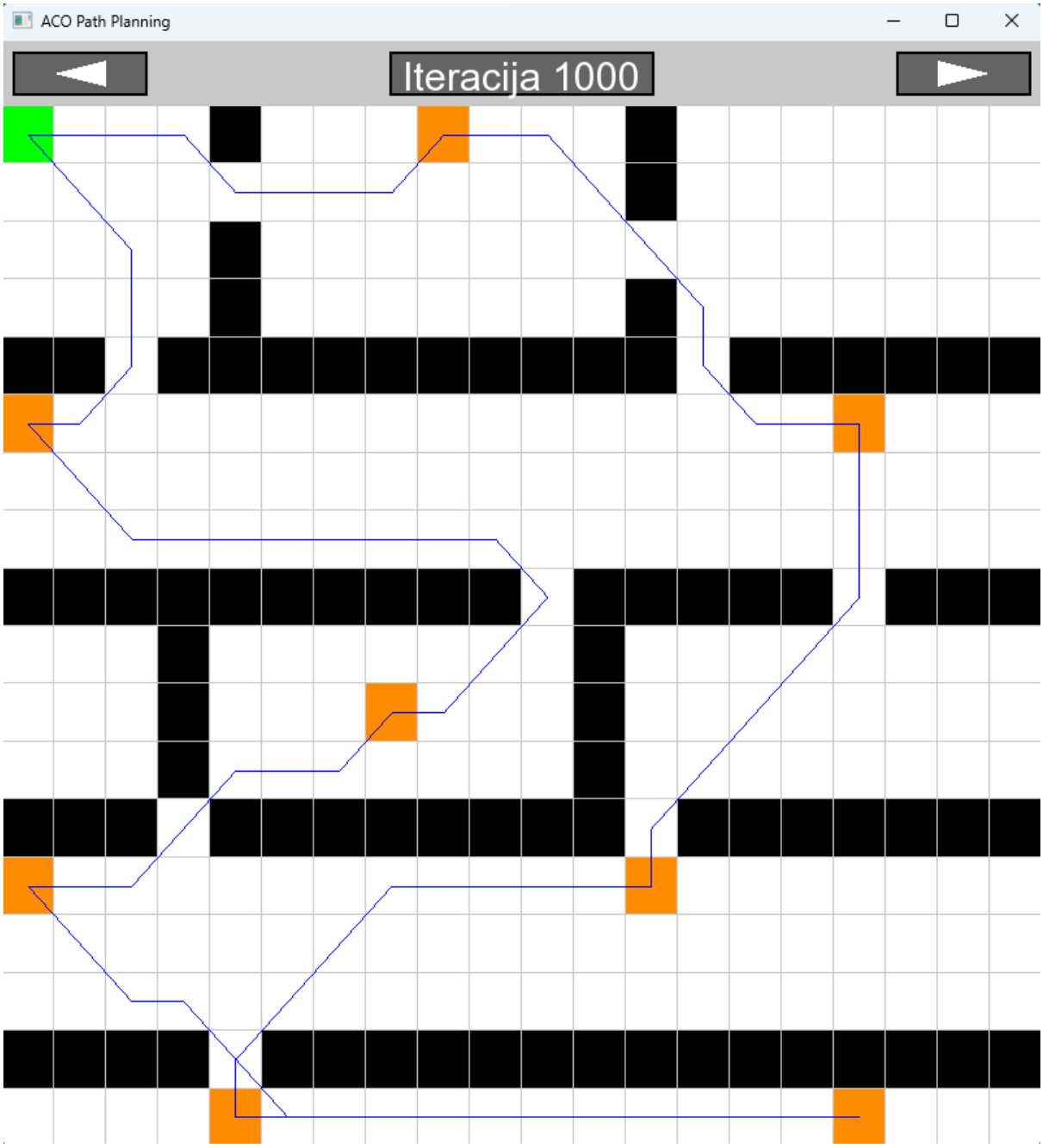
tako da bi vrednost $\frac{1}{Dužina\ putanje}$ sigurno bila manja od podrazumevanih 0.01, što znači, da bi se u tom slučaju količina položenog feromona bila ista za svaki pronađen put, čime bi usledio rezultat kao na slici:



- **Rezultati:** Vreme izvršavanja je značajno duže (~33 sekunde). Algoritam uspešno pronalazi validnu putanju kroz složeni lavirint. Iako je teško tvrditi da je putanja globalno optimalna bez poređenja sa egzaktnim algoritmom, pronađeno rešenje je veoma dobro i predstavlja logičan put od starta do cilja.

3.4 Multi-Waypoint TSP (20×18)

Četvrti eksperiment demonstrira ACO na NP-teškom problemu. Korišćena je mapa dimenzija 20×18 sa 8 tačaka prolaska i četiri horizontalne barijere koje stvaraju uska grla:



Naivni redosled obilaska ne može biti optimalan bez uzimanja u obzir topologije prepreka.

Parametri ACO-a: 80 mrava, 1000 iteracija, stopa isparavanja ϕ = 0,2, minimalni feromon τ_{min}= 0,001, *best-only* depozitovanje.

Algoritam deluje da uspešno konvergira ka rešenju koje grupiše geografski zavisne tačke prolaska, čime potvrđuje sposobnost ACO-a da efikasno rešava NP-teške probleme u realnom okruženju sa preprekama.

Mapa	Veličina	Korišćeni parametri	Cena najbolje putanje	Vreme izvršavanja (ms)
Jednostavna	10x10	iter=100, ants=64	~11.0711	~170
Srednja	20x20	iter=100, ants=64	~33.48	~320
Složena	40x40	iter=2000, ants=64, minimumPheromone=0.0001, evaporation=0.1	~106.9	~33000
TSP	20x18	Iter=1000, ants=80, evaporatoin=0.2, minimumPheromone=0.001	~100	~2000

4. Zaključak

U okviru ovog projekta uspešno je implementiran i testiran algoritam Optimizacije kolonijom mrava (ACO) za problem planiranja putanje robota. Rešenje je pokazalo sposobnost pronalaženja kvalitetnih putanja u statičkim 2D okruženjima sa preprekama, potvrđujući efikasnost ACO metaheuristike za ovu klasu problema. Interaktivni prikay sadrđaja se pokazao kao izuzetno koristan alat za analizu i razumevanje procesa konvergencije algoritma.

Pored pronalaženja najkraćeg puta, sistem podržava rešavanje TSP-a, gde ACO demonstrira sposobnost aproksimativne optimizacije nad prostorima pretrage faktorijalnog rasta.

Iako je rešenje funkcionalno i efikasno, postoje pravci za dalji razvoj:

1. **Poboljšanje performansi:** Trenutna implementacija je u potpunosti zasnovana na CPU. S obzirom na to da je kretanje svakog mrava nezavisan proces, algoritam je inherentno paralelan. Prebacivanje glavne petlje na GPU (npr. korišćenjem CUDA ili ROCm) bi drastično smanjilo vreme izvršavanja, što je ključno za primene u realnom vremenu ili na veoma velikim mapama. Ovo istraživanje je i započeto na drugoj grani.

2. **Podešavanje parametara:** Performanse ACO algoritma su osetljive na vrednosti parametara (α, β, evaporation, itd.). Automatsko podešavanje ovih parametara korišćenjem tehnika kao što su grid search ili genetski algoritmi moglo bi dodatno poboljšati kvalitet rešenja.

3. **Napredne ACO varijante:** Moguće je implementirati i druge, naprednije varijante ACO algoritma, kao što je *Max-Min Ant System (MMAS)*, koji ograničava minimalne i maksimalne vrednosti feromona kako bi se sprečila stagnacija i podstaklo bolje istraživanje prostora.

Sve u svemu, projekat predstavlja solidnu osnovu i demonstrira praktičnu primenljivost bio-inspirisanih algoritama u rešavanju složenih optimizacionih problema u robotici.

5. Literatura

https://www.sciencedirect.com/science/article/abs/pii/S0029801824020766

https://en.wikipedia.org/wiki/Ant_colony_optimization_algorithms

https://www.youtube.com/watch?v=u7bQomllcJw